Notes for Problem Solving in python

🚀 ملاحظات مهمة جدًا لحل المسائل بلغة بايثون 🐍🔥

أثناء حل المسائل في بايثون، في شوية تفاصيل ممكن متاخدش بالك منها، وعلشان كده جمعتلك **أهم الملاحظات والتريكات** اللي هتقابلك من شيت الأساسيات لحد شيت اللوبس! 📝💡

🎯 1. كيف تكتب `main` لتشغيل الكود؟ 🏁

في بايثون، الكود **مش محتاج** `main` علشان يشتغل، لكن كتابته بتنظم الكود، وبتخليك تفرق بين الـ **Functions** و **Classes**، وبتساعدك لما تعمل **Import للكود في ملف تاني**.

✅ **الصيغة الصح:**

```
if __name__ == '__main__':
```

⚡ **ليه نستخدم `main` ؟**

- تنظيم أفضل للكود 🏗️
- بيمنع تشغيل الكود تلقائيًا عند استيراد الملف 📂
- بيخلي الكود أكثر احترافية 🚀

⌨️ 2. ازاي تاخد إدخال من المستخدم؟ 📝

📌 للأسف دي كانت **سقطة غير مقصودة** في التلخيص السريع، لكن تعويضًا عنها، هشرحها بالتفصيل هنا! 🤩

📥 طريقة الإدخال الصحيحة في بايثون:

```
name_of_variable = input().strip()
```

- `input()` ⟵ بتاخد الإدخال من المستخدم كـ **String** 🎤
- `.strip()` ⟵ بيشيل أي مسافات **زيادة** في بداية أو نهاية الإدخال ✂️

✅ **مثال:**

```python
name = input("Enter your name: ").strip()
print(f"Hello, {name}! 👋")
```

🚀 3. الملاحظات اللي هتقابلك أثناء حل المسائل! 🔍

📌 استخدم `f-string` بدل الـ Concatenation ✨

لو المسألة بتطلب منك إن الـ **Output** يكون عبارة عن **كلام وقيمة معينة**، فالطريقة الأفضل **هي استخدام الـ** `f-string` بدل `+` لتنسيق الإخراج بشكل أنيق.

✅ **مثال:**

```python
a, b = 5, 10
print(f"{a} + {b} = {a + b} ")
```

🔹 **الناتج:**

```
5 + 10 = 15
```

⚡ **ليه نستخدم** `f-string` ؟
- ✔ أسرع من `format()` و `+` 🚀
- ✔ أوضح وأسهل في القراءة 👀
- ✔ بيساعد في تنسيق الأعداد بسهولة 🔢

📝 4. التعامل مع الإدخال على سطر واحد 🖥️

في بعض المسائل، لازم تاخد **أكثر من قيمة في سطر واحد**، لكن بايثون بيقرأ `input()` كـ **String واحد**، فالحل هنا هو `split()` ! 🚀

🔹 `split()` لتحويل الإدخال إلى List من القيم 📊

📌 لما تقسم الإدخال بـ `split()` , تقدر تخزن أكتر من قيمة **في متغير واحد** وتتعامل معاه كـ **Array (قائمة)**

✅ **مثال عملي:**

```python
if __name__ == '__main__':
    values = input().split()  # تقسيم الإدخال إلى قائمة قيم 📜
```

```
print(int(values[0]))   # أول قيمة كـ int 🔢
print(int(values[1]))   # ثاني قيمة كـ int 🔢
print(values[2])        # ثالث قيمة كـ String 🔤
print(float(values[3])) # رابع قيمة كـ float 🔢
print(float(values[4])) # خامس قيمة كـ float 🔢
```

📌 مثال على الإدخال

```
3 12345678912345 a 334.23 14049.30493
```

🔹 الإخراج المتوقع

```
3
12345678912345
a
334.23
14049.30493
```

🔥 5. ملاحظات مهمة عن التخزين في بايثون 💾

📌 **هل تعلم؟** 🤔 إن في بايثون، **المتغيرات الرقمية بتتخزن بطريقة مختلفة عن C++!**

في C++	في بايثون	المعلومة
`int`	`int`	في بايثون، **int بيشتغل زي** `long long` **في C++** 🏗️
`float`	`float`	في بايثون، **float بيتعامل زي** `double` **في C++** 💡

💡 معنى كده؟

- تقدر تخزن أرقام **كبيرة جدًا** بدون مشاكل زي `long long` في C++ 💪
- الحسابات العشرية هتكون بدقة عالية زي `double` في C++ 🎯

اعرف تفرق بين ال

Floor , Ceil , Round

1. دالة الفلور (Floor)

- بتجيب **أكبر عدد صحيح أقل من أو يساوي** الرقم اللي دخلته.
- يعني لو معاك عدد عشري، الفلور بيشيله ويجيب أقرب عدد صحيح قبله.

✅ **مثال:**

```
import math
print(math.floor(5.9))  # 5 :الناتج
print(math.floor(-2.3))  # 3- :الناتج
```

---

2. دالة السيل (Ceil)

- بتجيب **أصغر عدد صحيح أكبر من أو يساوي** الرقم اللي دخلته.
- يعني لو عندك عدد عشري، السيل هيزوده للي بعده على طول.

✅ **مثال:**

```
import math
print(math.ceil(5.1))  # 6 :الناتج
print(math.ceil(-2.7))  # 2- :الناتج
```

---

3. دالة الراوند (Round)

- بتجيب **أقرب عدد صحيح** للرقم اللي دخلته.
- لو العدد بين رقمين **هيقرب للأقرب**، ولو في النص بالظبط (.5) **هيقرب لأقرب رقم زوجي**.

✅ **مثال:**

```
print(round(5.4))  # 5 :الناتج
print(round(5.6))  # 6 :الناتج
print(round(2.5))  # 2 :الناتج (لأنه هيقرب لأقرب عدد زوجي)
print(round(3.5))  # 4 :الناتج (لأنه هيقرب لأقرب عدد زوجي)
```

ملخص سريع

العملية	بتعمل إيه؟	مثال	الناتج
فلور (`math.floor(x)`)	بيجيب أقرب عدد صحيح **أقل من أو يساوي** X	`math.floor(3.9)`	`3`
سيل (`math.ceil(x)`)	بيجيب أقرب عدد صحيح **أكبر من أو يساوي** X	`math.ceil(3.1)`	`4`
راوند (`round(x)`)	بيقرب X لأقرب عدد صحيح	`round(3.5)`	`4`
